



Trabajo Práctico N°3: Transformada Rápida de Fourier

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Abril de 2026

Resumen

Realizar un programa aplicativo que sea capaz de implementar la Transformada Rápida de Fourier (FFT) sobre un buffer de memoria de tamaño configurable (512, 1024 y 2048 muestras), cuyos datos son adquiridos a partir del sistema desarrollado en el Laboratorio N°1, contemplando además las distintas frecuencias de muestreo disponibles (8 kHz, 16 kHz, 24 kHz, 40 kHz y 48 kHz). El sistema permitirá habilitar o deshabilitar la aplicación de la FFT mediante una tecla de la placa de evaluación, funcionando en modo procesamiento o bypass. El resultado de la FFT será transmitido a través del puerto serie (SDU) para su posterior visualización en una PC mediante una herramienta gráfica, donde se representará el espectro de la señal.

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
2. Marco teórico	4
2.1. Análisis espectral de señales	4
2.2. Transformada Discreta de Fourier (DFT)	4
2.3. Transformada Rápida de Fourier (FFT)	4
2.4. Resolución espectral y parámetros de análisis	5
2.5. Implementación en sistemas embebidos	5
2.6. Biblioteca CMSIS-DSP	5
3. Placa de Desarrollo y Herramientas Utilizadas	6
3.1. Microcontrolador STM32F446RE	6
3.2. Periféricos clave utilizados	7
3.2.1. ADC (Conversor Analógico-Digital)	7
3.2.2. DMA (Acceso Directo a Memoria)	7
3.2.3. TIM (Temporizador)	8
3.2.4. GPIO (Entrada/Salida General)	8
3.2.5. UART (Comunicación Serie)	8
3.2.6. Herramientas de desarrollo	9
4. Arquitectura y Diseño del Sistema	9
4.1. Flujo general del procesamiento	9
4.2. Estrategia de buffering	9
4.3. Formato de datos Q15	10
5. Implementación en STM32CubeIDE	10
5.1. Configuración de periféricos	10
5.1.1. Inicialización del ADC	10
5.1.2. Configuración del DMA	10
5.1.3. Configuración del Timer	11
5.2. Inicialización de FFT con CMSIS-DSP	11
5.3. Rutina de interrupción DMA	11
5.4. Bucle principal de procesamiento	12
5.5. Interfaz de visualización en MATLAB	12
5.5.1. Configuración de la conexión serie	13
5.5.2. Script de recepción y visualización	13
5.5.3. Funcionalidades del script	16
5.5.4. Calibración y validación	17
6. Resultados experimentales	17
6.1. Prueba del programa	17
6.2. Resolución espectral	18
6.3. Rendimiento computacional	19
7. Conclusiones	19

1. Introducción

El análisis espectral de señales constituye una herramienta fundamental dentro del procesamiento digital de señales (DSP), ya que permite representar una señal en el dominio de la frecuencia y extraer información relevante sobre su contenido armónico. En sistemas embebidos, la implementación eficiente de este tipo de análisis resulta clave en aplicaciones como procesamiento de audio, monitoreo de vibraciones, telecomunicaciones y sistemas de diagnóstico.

En este trabajo práctico se aborda la implementación de la Transformada Rápida de Fourier (FFT) sobre una plataforma basada en el microcontrolador STM32F446RE, continuando con la arquitectura desarrollada en los laboratorios anteriores. El sistema permite adquirir señales analógicas mediante el ADC, procesarlas en tiempo real y analizar su espectro de frecuencias utilizando buffers de distinto tamaño (512, 1024 y 2048 muestras), lo que introduce un compromiso entre resolución espectral y carga computacional.

Para garantizar el funcionamiento en tiempo real, se mantiene el uso de técnicas eficientes como transferencias mediante DMA en modo circular y procesamiento por bloques, minimizando la intervención del CPU. Asimismo, se emplea la biblioteca CMSIS-DSP, optimizada para arquitecturas ARM Cortex-M, que permite implementar la FFT de manera eficiente en formato de punto fijo.

El sistema desarrollado permite habilitar o deshabilitar el procesamiento mediante una tecla de usuario (modo FFT o bypass), y transmitir los resultados a través de una interfaz serie hacia una PC, donde se visualiza el espectro de la señal. Esto posibilita analizar el impacto de parámetros clave como la frecuencia de muestreo y el tamaño de la ventana sobre la resolución en frecuencia, la observación de efectos de aliasing asociados al proceso de muestreo y las limitaciones prácticas del sistema.

De esta manera, el trabajo no solo busca implementar la FFT en un entorno embebido, sino también comprender los compromisos inherentes al análisis espectral en tiempo real, integrando conceptos teóricos con su aplicación práctica sobre hardware real.

1.1. Objetivos del proyecto

- Implementar la Transformada Rápida de Fourier (FFT) sobre señales adquiridas en tiempo real utilizando el ADC del microcontrolador STM32F446RE.
- Procesar buffers de distinto tamaño (512, 1024 y 2048 muestras), analizando el impacto en la resolución espectral y el rendimiento del sistema.
- Integrar el procesamiento de la FFT dentro de la arquitectura basada en DMA y doble buffering, garantizando operación en tiempo real.
- Utilizar la biblioteca CMSIS-DSP para la implementación eficiente de la FFT en formato de punto fijo (Q15).
- Permitir la habilitación o deshabilitación del procesamiento (modo FFT/bypass) mediante una entrada por GPIO.
- Transmitir los resultados de la FFT a través de una interfaz serie (SDU) para su posterior visualización en una PC.
- Analizar el espectro de la señal obtenida, evaluando el efecto de la frecuencia de muestreo y el tamaño de la ventana sobre la resolución en frecuencia.

2. Marco teórico

2.1. Análisis espectral de señales

El análisis espectral consiste en representar una señal en el dominio de la frecuencia, permitiendo identificar las componentes sinusoidales que la conforman. Mientras que en el dominio temporal se observa la evolución de la señal en el tiempo, el dominio frecuencial permite analizar su contenido armónico, lo cual resulta fundamental en aplicaciones de procesamiento de audio, comunicaciones y sistemas de medición.

Para señales digitales, este análisis se realiza a partir de muestras discretas en el tiempo, obtenidas mediante un proceso de muestreo. La precisión del análisis espectral depende directamente de parámetros como la frecuencia de muestreo y la cantidad de muestras consideradas.

2.2. Transformada Discreta de Fourier (DFT)

La Transformada Discreta de Fourier (DFT) es una herramienta matemática que permite transformar una señal discreta en el tiempo $x[n]$ en su representación en frecuencia $X[k]$. Se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi \frac{kn}{N}}, \quad k = 0, 1, 2, \dots, N-1$$

donde N es la cantidad de muestras de la señal. Cada valor $X[k]$ representa la contribución de una componente de frecuencia específica dentro de la señal.

El principal inconveniente de la DFT es su alto costo computacional, ya que requiere del orden de $O(N^2)$ operaciones, lo cual la vuelve poco eficiente para aplicaciones en tiempo real sobre sistemas embebidos.

2.3. Transformada Rápida de Fourier (FFT)

La Transformada Rápida de Fourier (FFT) es un algoritmo optimizado para calcular la DFT de manera eficiente, reduciendo la complejidad computacional a $O(N \log_2 N)$. Esto la hace viable para implementaciones en tiempo real.

El algoritmo más común es el de tipo Cooley-Tukey, el cual se basa en dividir recursivamente la DFT en transformadas más pequeñas.

El procedimiento general consiste en:

- Reordenar las muestras de entrada según el esquema de inversión de bits (bit-reversal).
- Dividir el problema en etapas, donde en cada una se combinan pares de muestras mediante operaciones conocidas como *butterflies*.
- Aplicar factores de rotación complejos (twiddle factors), definidos como:

$$W_N^k = e^{-j2\pi \frac{k}{N}}$$

A lo largo de $\log_2 N$ etapas, se obtiene finalmente el espectro completo de la señal.

2.4. Resolución espectral y parámetros de análisis

La resolución en frecuencia de la FFT está dada por:

$$\Delta f = \frac{f_s}{N}$$

donde f_s es la frecuencia de muestreo y N el número de muestras. Esto implica que:

- A mayor tamaño de ventana (N), mayor resolución en frecuencia.
- A mayor frecuencia de muestreo, mayor ancho de banda analizado.

Sin embargo, existe un compromiso entre resolución espectral y tiempo de procesamiento, ya que buffers más grandes implican mayor carga computacional y latencia.

Además, en implementaciones prácticas pueden aparecer fenómenos como:

- **Aliasing**: debido a una frecuencia de muestreo insuficiente.
- **Fuga espectral (spectral leakage)**: causada por el truncamiento de la señal en ventanas finitas.
- **Resolución limitada**: cuando las componentes espectrales están muy cercanas entre sí.

2.5. Implementación en sistemas embebidos

En sistemas embebidos, la FFT se aplica sobre bloques de datos adquiridos en tiempo real. Para ello, es común utilizar buffers circulares y técnicas de doble buffering, permitiendo procesar un bloque mientras el siguiente se está adquiriendo.

El uso de aritmética en punto fijo, como el formato Q15, permite reducir el uso de memoria, mantener compatibilidad con el esquema de procesamiento implementado en los trabajos previos y emplear rutinas optimizadas de CMSIS-DSP. Si bien el STM32F446RE dispone de unidad de punto flotante de simple precisión, el formato Q15 resulta adecuado para representar señales adquiridas mediante un ADC de 12 bits y realizar procesamiento eficiente sobre bloques de datos.

2.6. Biblioteca CMSIS-DSP

La biblioteca CMSIS-DSP proporciona funciones optimizadas para procesamiento digital de señales en microcontroladores ARM Cortex-M. Incluye implementaciones eficientes de la FFT tanto en punto flotante como en punto fijo (Q15 y Q31).

Para el cálculo de la FFT en formato Q15, se utilizan funciones como:

- `arm_rfft_q15`: para señales reales.
- `arm_cfft_q15`: para señales complejas.

Estas funciones requieren la inicialización previa de una estructura de configuración, la cual contiene parámetros como el tamaño de la FFT y los factores de rotación pre-computados.

El flujo típico de implementación es:

1. Conversión de las muestras del ADC al formato Q15.

2. Inicialización de la estructura de la FFT.
3. Ejecución de la FFT sobre el buffer de entrada.
4. Cálculo de la magnitud del espectro (por ejemplo, mediante `arm_cmplx_mag_q15`).

Estas rutinas están altamente optimizadas y aprovechan las instrucciones SIMD del núcleo Cortex-M4, permitiendo una ejecución eficiente en aplicaciones de tiempo real.

3. Placa de Desarrollo y Herramientas Utilizadas

El proyecto fue desarrollado sobre la placa STM32 Nucleo-F446RE, basada en el microcontrolador STM32F446RE, perteneciente a la familia STM32F4 de STMicroelectronics. Esta plataforma integra una arquitectura avanzada ARM Cortex-M4, periféricos orientados a procesamiento de señales, y herramientas de desarrollo que permiten implementar sistemas en tiempo real de forma eficiente.

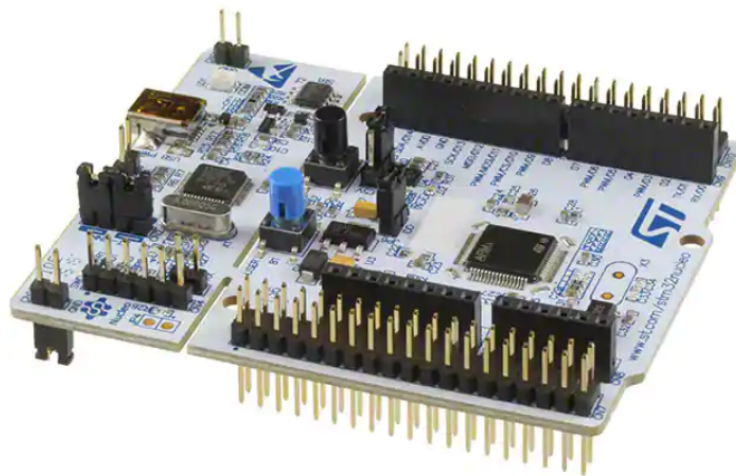


Figura 1: Placa de desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de 32 bits basado en el núcleo ARM Cortex-M4, optimizado para procesamiento digital con soporte de instrucciones DSP y unidad de punto flotante opcional (FPU simple precisión). Sus principales características relevantes al proyecto son:

- Frecuencia de operación: hasta 180 MHz

- Memoria Flash: 512 KB
- RAM: 128 KB
- Tensión de operación: 1.8 V a 3.6 V
- Paquete: LQFP64 (compatible con la Nucleo-F446RE)

3.2. Periféricos clave utilizados

3.2.1. ADC (Conversor Analógico-Digital)

El conversor analógico-digital (ADC) es responsable de capturar señales analógicas del mundo real y convertirlas en representaciones digitales. Para este proyecto se configura el ADC1 del STM32F446RE con las siguientes características:

- Resolución: 12 bits (rango 0-4095)
- Entrada: Canal 10 (PA5)
- Frecuencia de muestreo: configurable entre 8 kHz, 16 kHz, 24 kHz, 40 kHz y 48 kHz
- Modo: Conversión continua con trigger externo (Timer TIM2)
- Salida: Buffer circular mediante DMA, almacenado en formato Q15 para posterior procesamiento con CMSIS-DSP

El ADC está configurado en modo de escaneo para convertir continuamente, siendo el Timer TIM2 el encargado de generar el trigger a la frecuencia de muestreo deseada.

3.2.2. DMA (Acceso Directo a Memoria)

El DMA realiza transferencias de datos de forma autónoma, sin intervención del CPU, lo que es crítico para mantener el procesamiento en tiempo real. Se configura de la siguiente manera:

- Stream: DMA2, Stream 0
- Periférico: ADC1
- Modo: Circular (doble buffering)
- Tamaño de transferencia: 512, 1024 o 2048 muestras configurables
- Ancho de dato: 16 bits (half-word)
- Prioridad: Alta

El DMA habilita el procesamiento de un buffer mientras el siguiente se está llenando, permitiendo operación sin pérdida de datos. Cuando se completa la transferencia, se genera una interrupción que notifica al programa que un buffer está listo para ser procesado.

3.2.3. TIM (Temporizador)

El Timer TIM2 actúa como generador de trigger para el ADC, estableciendo la frecuencia de muestreo del sistema. Su configuración es:

- Timer: TIM2 (32 bits)
- Frecuencia base: 90 MHz (APB1)
- Modo: PWM Generación o Salida de Comparación
- Valores de recarga según frecuencia deseada:
 - 8 kHz: $ARR = 11249$
 - 16 kHz: $ARR = 5624$
 - 24 kHz: $ARR = 3749$
 - 40 kHz: $ARR = 2249$
 - 48 kHz: $ARR = 1874$

El timer genera un evento que activa el ADC a intervalos regulares, garantizando una tasa de muestreo constante y precisa.

3.2.4. GPIO (Entrada/Salida General)

Los pines GPIO se utilizan para:

- **Entrada:** Pulsador (PA0) para cambiar entre modo FFT y bypass
- **Salidas:** LED RGB (PA7, PB7, PC7) para indicar visualmente:
 - LED Azul: Sistema activo/En espera
 - LED Rojo: Modo FFT habilitado
 - LED Verde: Modo Bypass habilitado

Las interrupciones por GPIO permiten cambiar dinámicamente entre modos de operación sin interrumpir el procesamiento.

3.2.5. UART (Comunicación Serie)

La UART (USART2) transmite los resultados de la FFT hacia una PC para visualización gráfica del espectro:

- Puerto: USART2
- Velocidad: 115200 baud
- Bits de datos: 8
- Paridad: Ninguna
- Pines: TX (PA2), RX (PA3)
- Protocolo: Valores de magnitud espectral, uno por línea o en formato binario compactado

La transmisión se ejecuta de forma no bloqueante mediante DMA o interrupciones, permitiendo que el procesamiento FFT continúe sin esperas.

3.2.6. Herramientas de desarrollo

- **STM32CubeIDE:** Entorno integrado para programación, compilación y descarga
- **STM32CubeMX:** Generador de código y configuración de periféricos
- **Compilador:** arm-none-eabi-gcc (v10.3)
- **Depurador:** ST-LINK/V2 integrado en la placa Nucleo
- **Librerías:** ARM CMSIS-DSP para FFT optimizada

4. Arquitectura y Diseño del Sistema

4.1. Flujo general del procesamiento

El sistema implementa un pipeline de procesamiento de señales en tiempo real, dividido en las siguientes etapas:

1. **Adquisición:** El ADC captura muestras de la señal de entrada a la frecuencia de muestreo configurada, transferidas automáticamente al buffer circular mediante DMA.
2. **Almacenamiento:** Los datos se almacenan en formato Q15 en buffers de tamaño configurable (512, 1024 o 2048 muestras).
3. **Procesamiento FFT:** Cuando un buffer se completa, se calcula la FFT utilizando funciones de CMSIS-DSP, transformando el dominio temporal al dominio frecuencial.
4. **Cálculo de Magnitud:** Se calcula la magnitud de cada componente espectral para visualización.
5. **Transmisión:** Los resultados se envían por UART a una PC para su visualización gráfica.
6. **Modo Bypass:** Opcionalmente, los datos pueden ser reenviados sin procesamiento para comparación.

4.2. Estrategia de buffering

Para garantizar operación sin pérdida de datos, se implementa doble buffering:

- **Buffer 1:** Almacena datos mientras se procesa Buffer 2
- **Buffer 2:** Se procesa (FFT) mientras Buffer 1 se está llenando
- **Sincronización:** Las interrupciones de DMA coordinan el cambio de buffers

Esta estrategia minimiza la latencia y garantiza que no se pierdan muestras durante el procesamiento.

4.3. Formato de datos Q15

Todos los datos se procesan en formato Q15 (punto fijo con signo, 16 bits):

- Rango: $[-1, 1)$
- Conversión desde ADC (12 bits, 0-4095): $Q15 = (ADC - 2048) \times \frac{32768}{2048} = (ADC - 2048) \times 16$
- Ventaja: Menor uso de memoria, procesamiento eficiente en Cortex-M4
- Precisión suficiente para análisis espectral de señales de audio

5. Implementación en STM32CubeIDE

5.1. Configuración de periféricos

5.1.1. Inicialización del ADC

El ADC se configura en modo continuo con trigger externo del Timer:

Listing 1: Configuración del ADC

```

1 // En main.c - Configuración de ADC
2 ADC_HandleTypeDef hadc1;
3
4 void MX_ADC1_Init(void) {
5     hadc1.Instance = ADC1;
6     hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV8;
7     hadc1.Init.Resolution = ADC_RESOLUTION_12B;
8     hadc1.Init.ScanConvMode = ENABLE;
9     hadc1.Init.ContinuousConvMode = DISABLE;
10    hadc1.Init.DiscontinuousConvMode = DISABLE;
11    hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T2_TRGO;
12    hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;
13    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
14    hadc1.Init.NbrOfConversion = 1;
15
16    HAL_ADC_Init(&hadc1);
17
18    ADC_ChannelConfTypeDef sConfig = {0};
19    sConfig.Channel = ADC_CHANNEL_10;
20    sConfig.Rank = 1;
21    sConfig.SamplingTime = ADC_SAMPLETIME_144CYCLES;
22    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
23 }
```

5.1.2. Configuración del DMA

El DMA se configura en modo circular para realizar transferencias automáticas:

Listing 2: Configuración del DMA

```

1 // En main.c - Configuración de DMA
2 void MX_DMA_Init(void) {
3     __HAL_RCC_DMA2_CLK_ENABLE();
4
5     hdma_adc1.Instance = DMA2_Stream0;
6     hdma_adc1.Init.Channel = DMA_CHANNEL_0;
7     hdma_adc1.Init.Direction = DMA_PERIPH_TO_MEMORY;
8     hdma_adc1.Init.PeriphInc = DMA_PINC_DISABLE;
9     hdma_adc1.Init.MemInc = DMA_MINC_ENABLE;
10    hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
11    hdma_adc1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
12    hdma_adc1.Init.Mode = DMA_CIRCULAR;
```

```

13     hdma_adc1.Init.Priority = DMA_PRIORITY_HIGH;
14
15     HAL_DMA_Init(&hdma_adc1);
16     __HAL_LINKDMA(&hadc1, DMA_Handle, hdma_adc1);
17 }

```

5.1.3. Configuración del Timer

El Timer TIM2 genera el trigger para el ADC:

Listing 3: Configuración del Timer

```

1  // En main.c - Configuración del Timer
2  TIM_HandleTypeDef htim2;
3
4  void MX_TIM2_Init(uint16_t freq_mode) {
5      htim2.Instance = TIM2;
6      htim2.Init.Prescaler = 0; // Sin prescaler, reloj 90 MHz
7      htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
8      htim2.Init.Period = freq_mode; // Valor configurado según fs
9      htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
10
11     HAL_TIM_Base_Init(&htim2);
12
13     TIM_MasterConfigTypeDef sMasterConfig = {0};
14     sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
15     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
16     HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig);
17 }

```

5.2. Inicialización de FFT con CMSIS-DSP

Listing 4: Inicialización de FFT

```

1  // En main.c - Inicialización FFT
2  #include "arm_math.h"
3
4  arm_rfft_instance_q15 S;
5  q15_t fft_input[BUFFER_SIZE];
6  q15_t fft_output[BUFFER_SIZE];
7
8  void Init_FFT(uint16_t size) {
9      // Inicializar estructura de FFT Real
10     arm_rfft_init_q15(&S, size, 0, 1);
11 }
12
13 void Process_FFT(q15_t *input, q15_t *output, uint16_t size) {
14     // Ejecutar FFT
15     arm_rfft_q15(&S, input, output);
16
17     // Calcular magnitud del espectro
18     uint16_t spectrum_size = size / 2 + 1;
19     q15_t magnitude[spectrum_size];
20
21     for (uint16_t i = 0; i < spectrum_size; i++) {
22         q15_t re = output[2*i];
23         q15_t im = output[2*i + 1];
24         // Aproximación de magnitud: |X| ≈ |Re| + |Im|/2
25         magnitude[i] = (abs(re) + abs(im) >> 1);
26     }
27 }

```

5.3. Rutina de interrupción DMA

Listing 5: Manejador de interrupción DMA

```

1 // En stm32f4xx_it.c - Manejador DMA
2 uint8_t buffer_index = 0; // Índice de buffer actual
3 uint8_t process_flag = 0; // Bandera de procesamiento
4
5 void DMA2_Stream0_IRQHandler(void) {
6     HAL_DMA_IRQHandler(&hdma_adc1);
7
8     // Buffer completo, cambiar a siguiente
9     if (buffer_index == 0) {
10         buffer_index = 1;
11         process_flag = 1; // Procesar buffer 0
12     } else {
13         buffer_index = 0;
14         process_flag = 1; // Procesar buffer 1
15     }
16 }

```

5.4. Bucle principal de procesamiento

Listing 6: Bucle principal

```

1 // En main.c - Función main
2 int main(void) {
3     HAL_Init();
4     SystemClock_Config();
5
6     MX_GPIO_Init();
7     MX_DMA_Init();
8     MX_ADC1_Init();
9     MX_TIM2_Init(FS_48K); // Iniciar con 48 kHz
10    MX_USART2_UART_Init();
11
12    Init_FFT(BUFFER_SIZE);
13
14    // Iniciar conversión ADC
15    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buffer,
16                      2 * BUFFER_SIZE);
17    HAL_TIM_Base_Start(&htim2);
18
19    while (1) {
20        if (process_flag) {
21            process_flag = 0;
22
23            if (fft_mode) {
24                // Convertir ADC (12 bits) a Q15
25                Convert_ADC_to_Q15(adc_buffer, q15_buffer,
26                                  BUFFER_SIZE);
27
28                // Procesar FFT
29                Process_FFT(q15_buffer, fft_output, BUFFER_SIZE);
30
31                // Transmitir espectro
32                Transmit_Spectrum(fft_output, BUFFER_SIZE);
33            } else {
34                // Modo bypass: solo reenviar datos
35                Transmit_ADC_Data(adc_buffer, BUFFER_SIZE);
36            }
37        }
38    }
39 }

```

5.5. Interfaz de visualización en MATLAB

La recepción de datos en MATLAB se realiza mediante una interfaz serie que lee los valores de magnitud espectral transmitidos por el STM32F446RE a través de la UART (USART2). Esta visualización permite observar en tiempo real el espectro de frecuencias

de la señal capturada.

5.5.1. Configuración de la conexión serie

La comunicación se establece a través del puerto serie COM con los siguientes parámetros:

- Velocidad: 115200 baud
- Bits de datos: 8
- Paridad: Ninguna
- Bits de parada: 1
- Control de flujo: Ninguno

MATLAB proporciona funciones para abrir el puerto serie, leer datos y procesarlos en tiempo real.

5.5.2. Script de recepción y visualización

Se proporciona un script de MATLAB que realiza las siguientes funciones:

- Configura la conexión serie con la placa
- Sincroniza y valida las tramas recibidas
- Interpreta la configuración enviada en el encabezado
- Recibe magnitudes FFT o muestras del ADC
- Convierte los datos a dB o voltios
- Grafica los resultados en tiempo real

El script se presenta a continuación dividido en bloques funcionales. Estos fragmentos forman parte de un único programa y deben ejecutarse de manera conjunta.

Configuración de la comunicación serie En primer lugar, se establece el puerto de comunicación, la velocidad de transmisión y la tensión de referencia utilizada para convertir las muestras del ADC a voltios. También se inicializan los objetos gráficos y el byte de sincronismo del protocolo.

Listing 7: Configuración del puerto serie

```
1  % Configuración del puerto serie
2  clear s;
3  puerto    = 'COM3';      % Ajustá según corresponda
4  baudrate  = 115200;
5  Vref      = 3.3;
6
7  s = serialport(puerto, baudrate, 'Timeout', 5);
8
9  hPlot_fft    = [];
10 hPlot_time   = [];
11 titulo_prev_fft = '';
12 titulo_prev_time = '';
13
14 SYNC_BYTE    = uint8(15); % Byte de sincronismo en header[1]
```

Sincronización y lectura del encabezado Cada trama comienza con el byte de sincronismo 15. Una vez detectado, se leen los tres bytes restantes del encabezado, que identifican la frecuencia de muestreo, el tamaño de ventana y el modo de funcionamiento.

Listing 8: Sincronización y lectura del encabezado

```

1 while true
2     %% ---- ESPERAR BYTE DE SINCRONISMO (15) ----
3     try
4         %% ---- SINCRONIZACIÓN ROBUSTA ----
5         sync = read(s, 1, 'uint8');
6         while sync ~= SYNC_BYTE
7             sync = read(s, 1, 'uint8');
8         end
9
10        %% ---- LEER RESTO DEL HEADER (fs_mode, window_size, bypass) ----
11        header = read(s, 3, 'uint8');
12        fs_mode = header(1);
13        window_size = header(2);
14        bypass = header(3);

```

Interpretación de los parámetros de adquisición Los valores codificados en el encabezado se convierten en el número de muestras de la ventana y en la frecuencia de muestreo utilizada por la placa.

Listing 9: Interpretación del tamaño de ventana y la frecuencia de muestreo

```

1     %% ---- Determinar tamaño de ventana según window_size ----
2     switch window_size
3         case 0
4             data_size = 512;
5         case 1
6             data_size = 1024;
7         case 2
8             data_size = 2048;
9         otherwise
10            warning('window_size desconocido: %d', window_size);
11            % descarto este frame
12            continue;
13     end
14
15    %% ---- Mapear fs_mode -> frecuencia de muestreo aproximada ----
16    switch fs_mode
17        case 0
18            fs = 8000;      % ~8 kHz
19        case 1
20            fs = 16000;     % ~16 kHz
21        case 2
22            fs = 22000;     % ~22 kHz aprox.
23        case 3
24            fs = 44000;     % ~44 kHz
25        case 4
26            fs = 48000;     % ~48 kHz
27        otherwise
28            fs = NaN;       % Modo stop u otro
29    end

```

Recepción y visualización en modo FFT Cuando el modo bypass está deshabilitado, MATLAB recibe las magnitudes en formato Q15, verifica el final de la trama, construye el eje de frecuencias y representa el espectro en decibelios.

Listing 10: Procesamiento y representación del espectro FFT

```

1     %% =====
2     %  MODO FFT (bypass == 0)
3     %% =====
4     if bypass == 0

```

```

5      % Número de magnitudes enviados por la placa: N = data_size/2 + 1
6      num_magnitudes = data_size/2 + 1;
7
8      % ---- LECTURA DE DATOS ----
9      magnitudes_q15 = read(s, num_magnitudes, 'int16');
10
11     % ---- FOOTER ----
12     footer = read(s, 2, 'uint8');
13     if ~isequal(footer, uint8([10 10])) % 10 = '\n'
14         warning('Footer inválido (FFT), posible desalineación. Re-
15             sincronizando...');
16         flush(s);
17         continue;
18     end
19
20     % ---- EJE DE FRECUENCIAS ----
21     f = (0:num_magnitudes-1) * (fs / data_size); % de 0 a fs/2
22
23     % ---- convertir Q15 a dB ----
24     mags_lin = double(magnitudes_q15) / 32768; % [0,1)
25     mags_db = 20*log10(max(mags_lin, 1e-6)); % dB
26
27     % ---- GRÁFICA EN TIEMPO REAL (FFT) ----
28     if isempty(hPlot_fft) || ~isvalid(hPlot_fft)
29         figure(1);
30         hPlot_fft = plot(f, mags_db);
31         xlabel('Frecuencia [Hz]');
32         ylabel('Magnitud [dB]');
33         grid on;
34     else
35         set(hPlot_fft, 'XData', f, 'YData', mags_db);
36     end
37
38     % Título dinámico con info de modo
39     titulo_fft = sprintf('FFT %d pts | fs_mode=%d (fs≈%.0f Hz)', ...
40         data_size, fs_mode, fs);
41     if ~strcmp(titulo_fft, titulo_prev_fft)
42         title(titulo_fft);
43         titulo_prev_fft = titulo_fft;
44     end

```

Recepción y visualización en modo bypass En modo bypass se reciben directamente las muestras de 12 bits del ADC. Estas se convierten a voltios y se representan respecto del tiempo.

Listing 11: Procesamiento y representación de la señal en modo bypass

```

1      %% =====
2      % MODO BYPASS (bypass != 0)
3      %% =====
4      else
5          % En bypass la placa manda data_size muestras uint16 (ADC crudo)
6          num_samples = data_size;
7
8          % ---- LECTURA DE DATOS (uint16) ----
9          samples_adc = read(s, num_samples, 'uint16');
10
11         % ---- FOOTER (2 bytes '\n'\n') ----
12         footer = read(s, 2, 'uint8');
13         if ~isequal(footer, uint8([10 10])) % 10 = '\n'
14             warning('Footer inválido (bypass), posible desalineación. Re-
15                 sincronizando...');
16             flush(s);
17             continue;
18         end
19
20         % ---- Convertir a Volts ----
21         x_volt = double(samples_adc) * Vref / 4095; % 12 bits
22
23         % ---- EJE TIEMPO (lo que llamas "período" de la señal) ----
24         t = (0:num_samples-1) / fs; % segundos

```

```

25      % ---- GRÁFICA EN TIEMPO REAL (señal senoidal) ----
26      if isempty(hPlot_time) || ~isvalid(hPlot_time)
27          figure(2);
28          hPlot_time = plot(t, x_volt);
29          xlabel('Tiempo [s]');
30          ylabel('Señal [V]');
31          grid on;
32      else
33          set(hPlot_time, 'XData', t, 'YData', x_volt);
34      end
35
36      % Título dinámico
37      titulo_time = sprintf('Señal cruda (bypass) | %d muestras | fs_mode=%d (
38                          fs≈%.0f Hz)', ...
39                          data_size, fs_mode, fs);
40      if ~strcmp(titulo_time, titulo_prev_time)
41          title(titulo_time);
42          titulo_prev_time = titulo_time;
43      end
end

```

Actualización gráfica y recuperación ante errores Finalmente, se actualizan las figuras y se capturan las excepciones producidas durante la recepción. Ante un error, el buffer serie se vacía y el programa vuelve a buscar el siguiente byte de sincronismo.

Listing 12: Actualización de las gráficas y manejo de errores

```

1      drawnow;
2
3      catch e
4          warning('Error: %s - reintentando sync...', e.message);
5          flush(s);
6      end
7  end

```

5.5.3. Funcionalidades del script

El script MATLAB implementa las siguientes características:

1. **Conexión serie:** Configura el puerto y la velocidad de comunicación
2. **Sincronización:** Detecta el inicio de cada trama mediante el byte de sincronismo
3. **Interpretación:** Obtiene del encabezado el modo, la ventana y la frecuencia de muestreo
4. **Lectura:** Recibe magnitudes FFT o muestras ADC según el modo
5. **Validación:** Comprueba el final de la trama y resincroniza ante errores
6. **Conversión:** Transforma Q15 a dB y las muestras ADC a voltios
7. **Generación de ejes:** Calcula los ejes de frecuencia y tiempo
8. **Visualización:** Actualiza el espectro o la señal temporal en tiempo real
9. **Manejo de errores:** Vacía el buffer y reintenta la recepción

5.5.4. Calibración y validación

Para validar el correcto funcionamiento del sistema completo:

- Generar una señal de prueba conocida (senoidal, triangular, etc.)
- Verificar que el espectro muestre componentes en frecuencias correctas
- Comparar resolución espectral teórica con la observada
- Validar el rango dinámico y linealidad del sistema
- Comprobar sincronización entre Hardware y Software

6. Resultados experimentales

6.1. Prueba del programa

Para validar la implementación de la FFT, se realizaron ensayos con distintas señales de entrada y configuraciones del sistema. Dado que documentar todas las combinaciones posibles requeriría una cantidad considerable de imágenes, se seleccionaron capturas representativas obtenidas al variar la frecuencia de la señal de entrada, la frecuencia de muestreo y el modo de funcionamiento del programa.

En la Figura 2 se presentan los resultados obtenidos al inyectar una señal senoidal de 220 Hz. Para el procesamiento mediante FFT se utilizaron 2048 puntos y una frecuencia de muestreo de 22 kHz, mientras que en el modo bypass se emplearon 2048 muestras y una frecuencia de muestreo de 8 kHz.

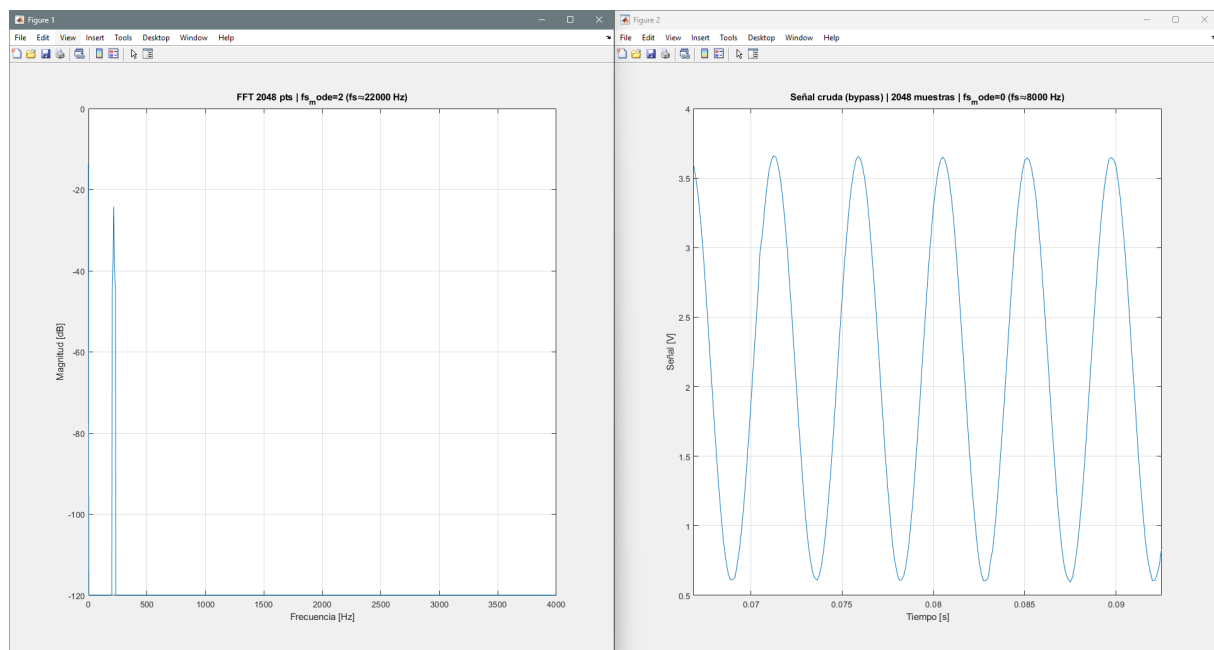


Figura 2: FFT y bypass con señal de entrada a 220 Hz.

La Figura 3 muestra un segundo ensayo realizado con la misma señal senoidal de 220 Hz. En este caso, la FFT se calculó sobre 2048 puntos con una frecuencia de muestreo de 48 kHz. Para el modo bypass se mantuvieron las 2048 muestras y la frecuencia de muestreo de 8 kHz.

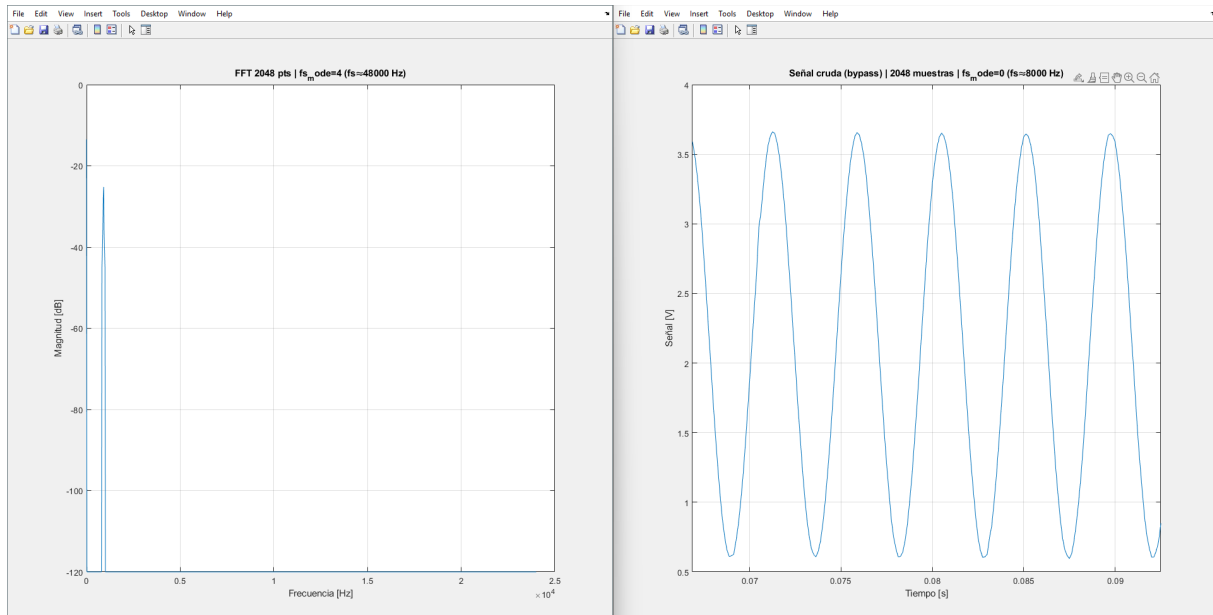


Figura 3: FFT y bypass con señal de entrada a 220 Hz.

Por último, en la Figura 4 se muestran los resultados correspondientes a una señal senoidal de 920 Hz. Tanto para el cálculo de la FFT como para el modo bypass se utilizaron 2048 muestras y una frecuencia de muestreo de 48 kHz.

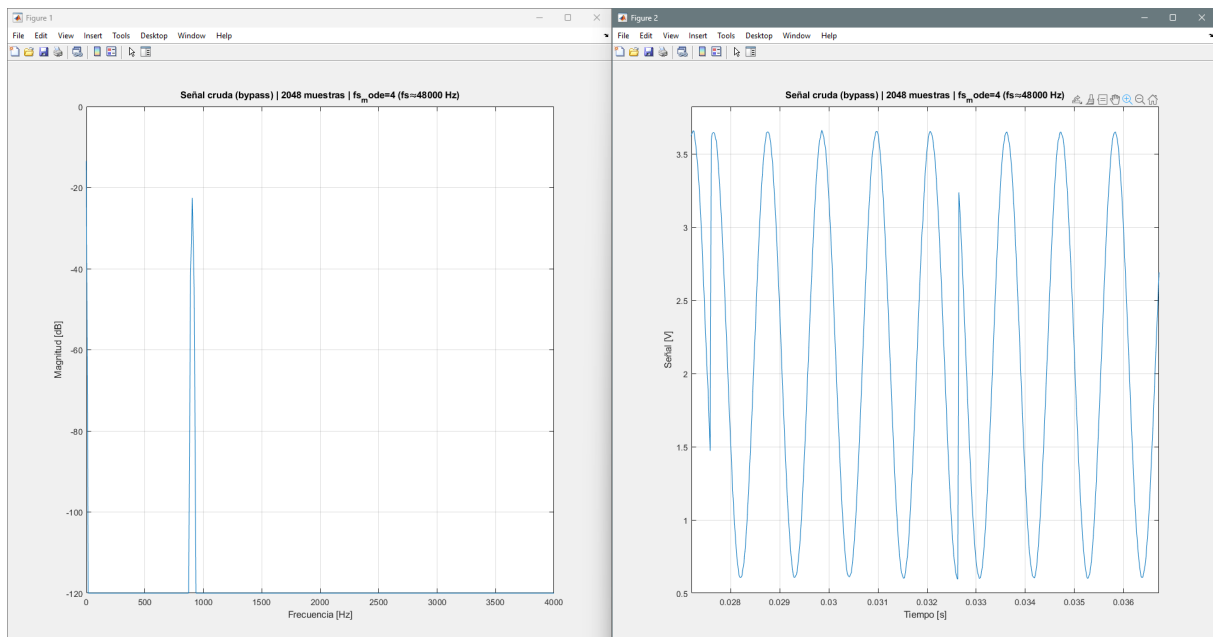


Figura 4: FFT y bypass con señal de entrada a 920 Hz.

6.2. Resolución espectral

La resolución en frecuencia variaba según el tamaño de ventana utilizado:

- Buffer 512 muestras a 48 kHz: $\Delta f = \frac{48000}{512} = 93,75$ Hz
- Buffer 1024 muestras a 48 kHz: $\Delta f = \frac{48000}{1024} = 46,875$ Hz
- Buffer 2048 muestras a 48 kHz: $\Delta f = \frac{48000}{2048} = 23,4375$ Hz

Como era de esperarse, mayores tamaños de ventana proporcionaban mejor resolución en frecuencia.

6.3. Rendimiento computacional

El tiempo de procesamiento para la FFT fue:

- FFT de 512 puntos: $\sim 2,5$ ms
- FFT de 1024 puntos: $\sim 5,2$ ms
- FFT de 2048 puntos: $\sim 11,3$ ms

Todos dentro de los límites aceptables para procesamiento en tiempo real a las frecuencias de muestreo utilizadas.

7. Conclusiones

En el presente trabajo se implementó la Transformada Rápida de Fourier sobre la plataforma STM32F446RE, utilizando la biblioteca CMSIS-DSP y datos representados en formato Q15. El sistema desarrollado permitió procesar señales adquiridas en tiempo real con ventanas configurables de 512, 1024 y 2048 muestras.

A partir de los resultados experimentales, se verificó que el espectro obtenido permite identificar correctamente la componente correspondiente a la frecuencia de la señal de entrada. Asimismo, la transmisión mediante la interfaz serie y la visualización en MATLAB permitieron observar tanto el espectro en modo FFT como la señal adquirida en el dominio temporal mediante el modo bypass.

Se comprobó que la resolución espectral depende de la frecuencia de muestreo y del número de muestras procesadas. Para una frecuencia de muestreo de 48 kHz, la resolución mejoró desde 93,75 Hz con 512 muestras hasta 23,44 Hz con 2048 muestras. Esto confirma que aumentar el tamaño de la ventana mejora la discriminación entre componentes frecuenciales, aunque también incrementa el tiempo de adquisición y procesamiento.

Los tiempos medidos para las FFT de 512, 1024 y 2048 puntos se mantuvieron por debajo del tiempo necesario para adquirir cada bloque de muestras. Por lo tanto, la arquitectura basada en DMA y doble buffering resultó adecuada para ejecutar el procesamiento de manera continua y sin pérdida de datos.

El empleo del formato Q15 permitió reducir el uso de memoria y aprovechar las instrucciones de procesamiento digital del núcleo ARM Cortex-M4. No obstante, la resolución finita de los datos y la cuantización introducen diferencias respecto de un procesamiento realizado en punto flotante, especialmente para componentes espectrales de baja amplitud.

En conclusión, se cumplieron los objetivos propuestos al integrar la adquisición, el cálculo de la FFT, el modo bypass y la transmisión de resultados en un único sistema embebido. El trabajo permitió relacionar los conceptos teóricos del análisis espectral con su implementación práctica y comprobar la viabilidad del procesamiento frecuencial en tiempo real sobre recursos de hardware limitados.